**SMARTBRIDGE**
Let's bridge the gap

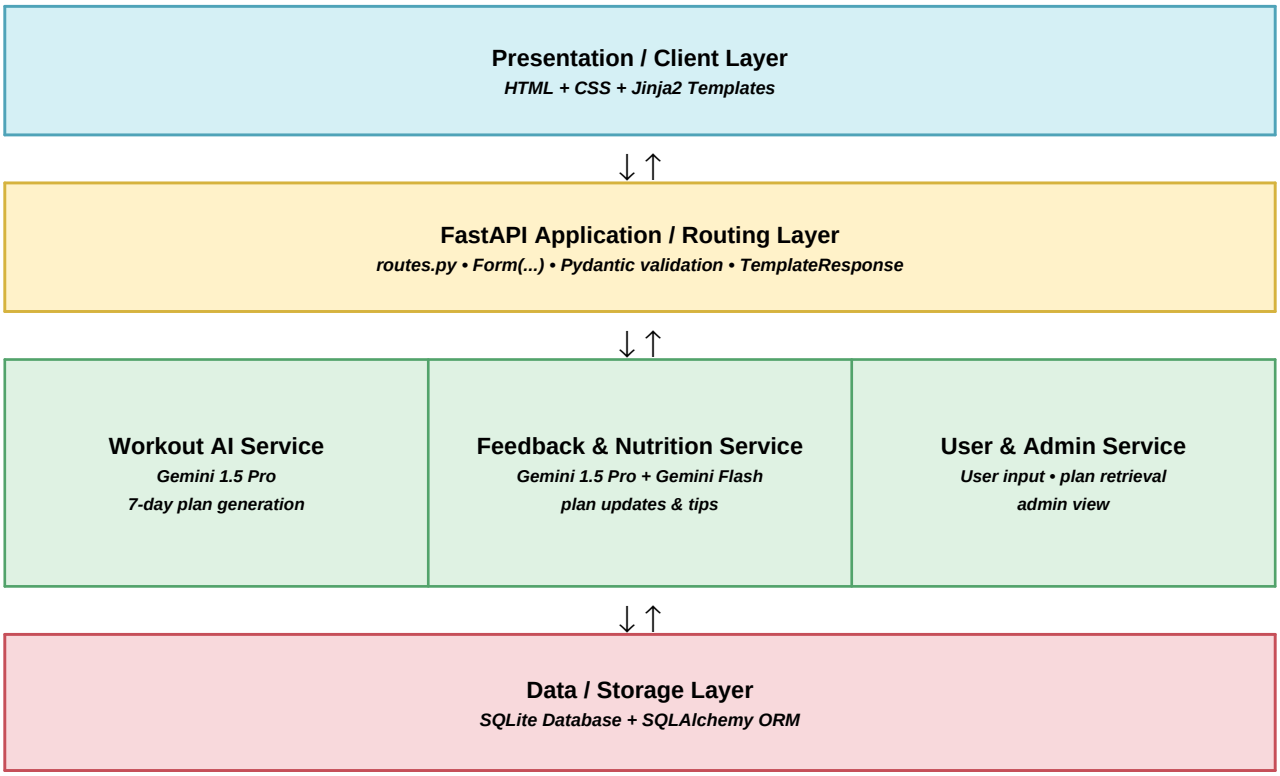
**SKILL WALLET**
A SmartBridge Project

# Solution Architecture

| | |
|---|---|
| **Date** | 01 October 2026 |
| **Team ID** | SWTID-2026-6912 |
| **Project Name** | FitBuddy - AI Fitness Plan Generator using Gemini Models |
| **Maximum Marks** | 5 Marks |

### Solution Architecture Diagram:

FitBuddy follows a modular FastAPI-based architecture with an HTML/Jinja2 presentation layer, FastAPI routing and processing, Gemini AI services, and SQLite/SQLAlchemy persistence. The structure below maps the technologies and components described in the project document.




**Primary data flow:** User input → FastAPI routes → Gemini-powered generation/update → SQLite persistence → Jinja2-rendered result. The project also provides an admin view of registered users and their original/updated plans.

# Component Description Table

| Component Name | Description / Role in Architecture | Technologies Used |
|---|---|---|
| **Presentation Layer** | Collects user details and displays personalized workout plans, nutrition tips and feedback forms. The admin page displays registered users and their plans. | HTML, CSS, Jinja2 |
| **FastAPI / Routing Layer** | Handles form input, validation, route processing, AI function calls and dynamic template responses. Main routes include /, /generate-workout, /submit-feedback and /view-all-users. | FastAPI, Uvicorn, Pydantic |
| **Workout AI Service** | Generates structured day-wise 7-day workout plans using user goal and intensity, including warm-up, main workout and cooldown/recovery guidance. | Google Gemini 1.5 Pro |
| **Feedback & Nutrition Service** | Uses feedback and the original plan to create revised plans, and produces concise nutrition/recovery tips aligned with the selected goal. | Gemini 1.5 Pro, Gemini Flash |
| **User & Admin Service** | Processes user information, retrieves plans and supports the administrator's view of users, original plans and updated plans. | FastAPI, Jinja2 |
| **Data / Storage Layer** | Stores user details, generated plans and feedback-based updated plans. Original and updated plans are kept separately for comparison. | SQLite, SQLAlchemy ORM |

## Key API Routes

| Route | Function |
|---|---|
| / | Displays index.html and collects user input. |
| /generate-workout | Processes user details, generates the workout plan and nutrition tip, saves the user and plan, and renders result.html. |
| /submit-feedback | Retrieves the original plan, sends plan + feedback to the AI update function, stores the revised plan and renders the updated result. |
| /view-all-users | Retrieves users and plans and displays them through all_users.html for the admin view. |

**SMARTBRIDGE**
Let's bridge the gap

**SKILL WALLET**
A SmartBridge Project

# Team Work Allocation

Responsibilities are assigned according to the exact order of names provided and mapped to the FitBuddy architecture and project workflow documented in the source project report.

| S.No | Team Member | Assigned Work | Architecture / Milestone |
|---|---|---|---|
| 1 | **Pavithra M** | **Model Selection & Architecture**<br>Research and select the Gemini models, define the modular application architecture, and support development-environment setup. | AI Layer + Architecture<br>Milestone 1 |
| 2 | **Manisha S** | **Core AI Functionality**<br>Develop personalized 7-day workout generation with Gemini 1.5 Pro, nutrition/recovery tips with Gemini Flash, and feedback-based plan updating. | Workout AI + Feedback/Nutrition<br>Milestone 2 |
| 3 | **Monica G** | **Backend, Routing & Database**<br>Implement FastAPI routing, Form/Pydantic input processing, routes.py logic, and SQLite/SQLAlchemy functions for users and plans. | FastAPI + Data Layer<br>Milestones 2 & 3 |
| 4 | **Monisha Anandan** | **Frontend & Jinja2 Templates**<br>Develop index.html, result.html and all_users.html, responsive CSS, dynamic rendering and user/admin interaction pages. | Presentation Layer<br>Milestone 4 |
| 5 | **Mridula M** | **Deployment, Testing & Documentation**<br>Prepare local deployment, configure environment variables, verify application routes and user flows, and document testing/deployment results. | Deployment & Verification<br>Milestone 5 |

## Architecture-to-Team Mapping

| Layer / Component | Responsible Member | Main Deliverables |
|---|---|---|
| Presentation / Client Layer | Monisha Anandan | index.html, result.html, all_users.html, CSS, Jinja2 rendering |
| FastAPI / Routing Layer | Monica G | routes.py, form handling, Pydantic validation, API route integration |
| Workout AI Service | Manisha S | generate_workout_gemini(), Gemini Pro prompts and 7-day plan generation |
| Feedback & Nutrition Service | Manisha S | update_workout_plan(), generate_nutrition_tip_with_flash() |
| User & Admin Service | Monica G | User/plan retrieval, save/update functions and admin data flow |
| Data / Storage Layer | Monica G | SQLite database, SQLAlchemy ORM, original/updated plan storage |
| Deployment & Verification | Mridula M | Local Uvicorn deployment, testing, environment configuration and documentation |
| Architecture & Model Selection | Pavithra M | Gemini selection, architecture definition and development setup |

**Source basis:** FitBuddy's documented architecture uses HTML/Jinja2 for the frontend, FastAPI for backend routing and logic, Gemini Pro and Flash for AI functionality, and SQLite with SQLAlchemy for persistence. The project workflow covers model selection and architecture, core functionality, routes.py development, frontend development and deployment.